CHAPTER 1

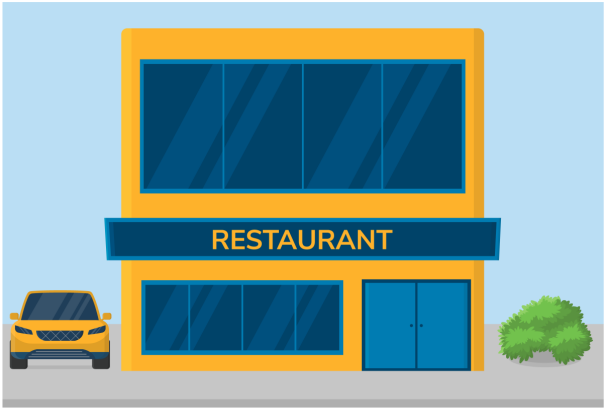
DESIGNING A RESTAURANT MANAGEMENT SYSTEM

1 Getting Ready: The Restaurant Management System 2
2 Requirements for the Restaurant Management System 3
3 Use Case Diagram for the Restaurant Management System 5
4 Class diagram for the Restaurant Management System 8
5 Sequence Diagram for the Restaurant Management System 18
6 Activity Diagram for the Restaurant Management System 19
7 Code of Restaurant Management System 23
8 Summary: Designing a Restaurant Management System 30
9 Key Insights: The Restaurant Management System 34

This chapter explores how to design a restaurant management system, covering requirements, UML diagrams, and implementation code for the system.

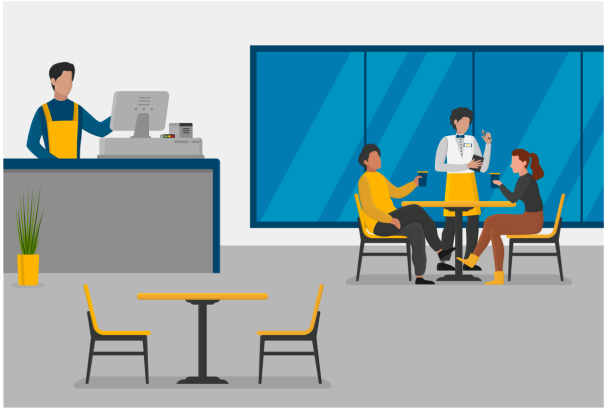
1 Getting Ready: The Restaurant Management System

A **restaurant management system** is a software platform used to manage various aspects of a restaurant's operations, such as food and beverage sales, table reservations, billing, and seating arrangements. It supports various restaurant types—from quick-service outlets to fine dining—and helps streamline service and improve customer satisfaction.
The system allows managers to monitor available tables, handle reservations, and generate bills—all from a central interface.



(a) An outer view of the restaurant

(b) The food being prepared for the customers



(a) A service being provided to the customers

1.1 Expectations from the interviewee

A restaurant management system has several components, each with specific constraints and requirements. The following provides an overview of some of the main expectations that the interviewer will want to hear you discuss in more detail during the interview.

Restaurant services

To better understand the services offered by a restaurant management system, you may ask the interviewer the following questions:

1. Does the restaurant provide a delivery service?
2. Can a customer place an online order?

3. Does the restaurant accept online/card payments?

Restaurant management

Ask these to clarify structural and operational needs:

- Can the restaurant have multiple branches?
- Do we need to account for inventory management?

1.2 Design approach

We'll design this restaurant management system using the bottom-up approach. Therefore, we'll follow the steps below:

- Identify and design the smallest components—the table, table seat, meal, meal item, and seating chart.
- Use these small components to design bigger components—the menu, branch, and restaurant.
- Repeat the steps above until we design the whole restaurant management system.

1.3 Design pattern

During an interview, discussing the design patterns under which the restaurant management system falls is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:

Which design pattern(s) should be used to design a restaurant management system? Please elaborate on your choice(s).

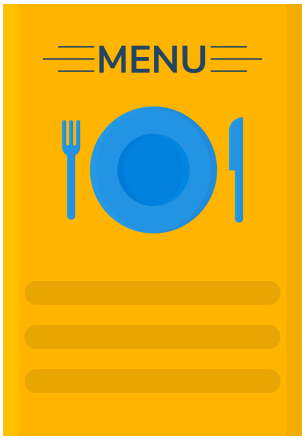
Let's explore the requirements of the restaurant management system in the next lesson.

2 Requirements for the Restaurant Management System

In this lesson, we'll list the restaurant management system design requirements. This is a crucial step since requirements define the scope of a problem. Therefore, getting them right from the interviewer and understanding them well will make the design of the rest of the system smooth and easy.
We'll use the notational convention to identify each requirement with a unique label, "Rn," where "R" is short for requirement and "n" is a natural number.

2.1 Requirement collection

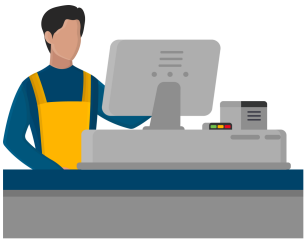
R1: The restaurant can have multiple branches.
R2: Each branch will offer a menu with various sections and items.
R3: The server should be able to create an order for a table, add items for each person seated, and update the order as needed.



R4: Each person's order can consist of multiple items, each corresponding to a menu item.
R5: The system should be able to provide information about tables currently available for walk-in customers.
R6: The system should allow for the reservation of tables.

R7: The receptionist should be able to search for available tables by date and time and make a reservation.
R8: The system should allow customers to make and cancel reservations.
R9: The system should send notifications as the reservation time approaches.

R10: Customers should be able to pay their bills with credit cards or cash.
R11: Each branch may have a different configuration of tables, managed by the branch manager. The manager can also update the branch's menu.
R12: The system shall allow the server to initiate the payment process for a generated bill by selecting a payment method (credit card or cash), after which the customer completes the payment.



We've identified our requirements for the problem. In the next lesson, we'll define different use cases of the restaurant management system.

3 Use Case Diagram for the Restaurant Management System

Let's build the use case diagram for the restaurant management system and understand the relationship between its different components. First , we'll define the different elements of our restaurant, followed by the complete use case diagram of the system.

3.1 System

Our system is a restaurant management system.

3.2 Actors

Let's define the main actors of our restaurant management system.

Primary actors

- **Customer:** This is the restaurant's primary actor who can view the menu, place orders, make a reservation, and make payments.
- **Receptionist:** This actor is responsible for reserving tables and updating the table reservation status.
- **Server:** This actor takes the order from the customer and processes the payment.
- **Manager:** This actor manages each branch's menu and table configuration.

Secondary actor

- **System:** This can automatically notify customers when their reservation time approaches and update table statuses whenever a bill is paid or a reservation is made.

3.3 Use cases

This section will define the restaurant management system's use cases. We 've listed them according to their respective interactions with a particular actor.

Customer

- **View menu:** To browse food items and prices.
- **Make reservation:** To book a table.
- **Cancel reservation:** To cancel an existing reservation.
- **Place order:** To order food.
- **View order:** To check order status or details.
- **Receive notification:** To receive a notification when the reservation time approaches.
- **Pay bill:** To complete payment using cash or a card.
- **Search available tables:** To check table availability in real time.
- **Request Bill Details:** To get the details to pay the bill.

Manager

- **Add/modify menu section:** To manage food categories.
- **Add/modify menu item:** To manage individual food items.
- **Add/update table configuration:** To define branch-specific table layouts.
- **View menu:** To access the menu for decision-making or management.

Receptionist

- **Search available tables:** To check table availability in real time.
- **Make reservation:** To book a table for a customer.
- **Cancel reservation:** To cancel an existing reservation.
- **Receive notification:** To receive notification when a reservation time approaches.
- **Modify reservation:** To modify an existing reservation.

Server

- **View menu:** To assist with customer ordering.
- **Take order:** To take and enter customer orders.
- **Fetch order list:** To view assigned or active orders.
- **Replace meal item:** To substitute an item in an existing order.
- **Generate bill:** To calculate the total bill against the food order.

System

- **Send notifications:** To automatically notify customers when their reservation time approaches.
- **Update table statuses:** To handle automatic updates of table availability.

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Associations

The table below shows the association relationship between actors and their use cases.

Customer	Receptionist	Server	Manager
View menu	Search available tables	View menu	Add/modify menu section
Make reservation	Make reservation	Take order	Add/modify menu item
Pay bill via cash/card	Update/cancel reservation	Fetch order list	Add/update table configuration
Cancel reservation	Receive notification	Replace meal item	
Place order		Generate bill	
View order			
Receive notification			
Search available tables			
Request Bill Details			

Include

When a customer wants to make a reservation, the system checks for available tables before confirming the booking. Additionally , the system updates the table status once the reservation is made.

- The **Make reservation** use case includes **search available tables** and **update table status** use cases**.**

When a customer cancels their reservation, the system updates the table status to reflect the availability.

- The **Cancel reservation** use case includes **Update table status** use case.

When staff take a customer's order, they must first view the current menu to select items.

- The **Take order** use case includes **View menu** use case.

When a customer pays the bill, the system marks the table as paid and updates its status accordingly.

- The **Pay bill** use case includes **Update table status** use case.

Extend

Users can pay using cash or a card when they choose to process a payment.

- The **Process payment** use case extends to **Pay bill via cash** and **Pay bill via card** use case.

When a user wants to change the details of an existing reservation, they modify the original booking.

- The **Modify reservation** use case extends the **Make reservation** use case.

3.5 Use case diagram

Here's the use case diagram of the restaurant management system:

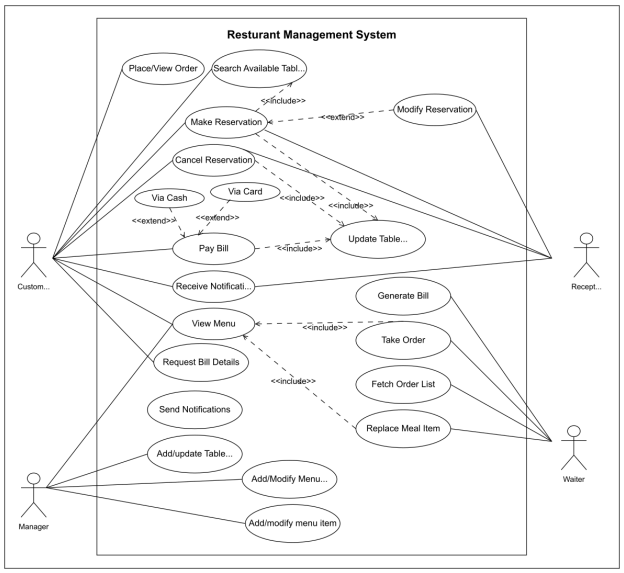


Figure 1.3: The use case diagram of the restaurant management system

In the next lesson, we'll discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class diagram for the Restaurant Management System

We'll create the class diagram for the restaurant management system. In the class diagram, we will first design the classes and then identify the relationships between classes according to the requirements of the restaurant management system design problem.

4.1 Components of the restaurant management system

As mentioned, we'll follow the bottom-up approach to design a class diagram for the restaurant management system.

Person

The `Person` class is extended by the `Waiter`, `Receptionist`, `Manager`, and `Customer` classes and stores the person's name, email, and phone number.

- `Waiter`: Takes an order from the customer and generates the bill.
- `Receptionist`: Creates a reservation.
- `Manager`: Can update the seating plan and can also update the menu.
- `Customer`: This derived class can make reservations, place orders, and pay bills.

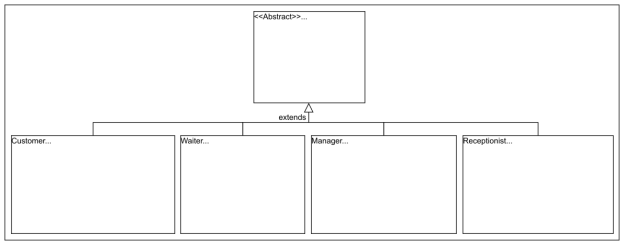


Figure 1.4: Person and its derived classes

R3: The server should be able to create an order for a table, add items for each person seated, and update the order as needed.
R6: The system should allow for the reservation of tables.
R7: The receptionist should be able to search for available tables by date and time and make a reservation.

Table and table seat

The `Table` class is identified by an ID, a limited seating capacity, a specific location, and a status. A table may have multiple seats.

R5: The system should be able to provide information about tables currently available for walk-in customers.
R6: The system should allow for the reservation of tables.

Meal and meal item

The `Meal` class represents a customer's meal at a specific seat and table and contains a list of ordered items.
`MealItem`: It is identified by a meal item ID and has a specific quantity that can be updated.

R4: Each person's order can consist of multiple items, each corresponding to a menu item.

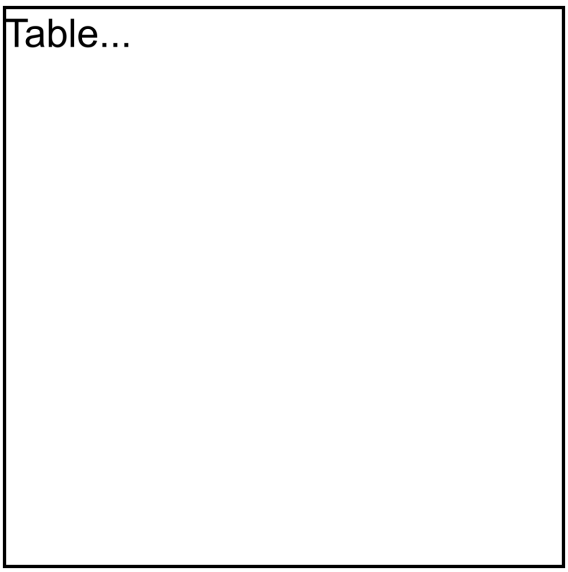


Figure 1.5: The Table class

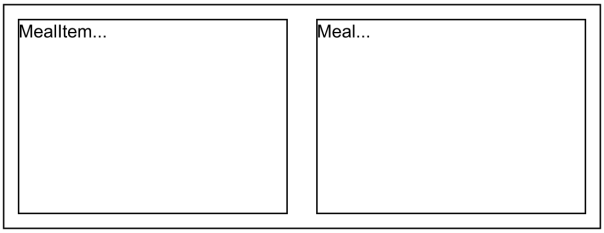


Figure 1.6: The Meal and MealItem classes

Menu, menu section, and menu item

`Menu`**:** It has a unique ID, title, and description, and may have several sections.
`MenuSection`**:** This class represents a menu section with an ID, a title, and a description. It may have multiple menu items.
`MenuItem`**:** It is identified by an ID and has a title, description, and price that can be updated.

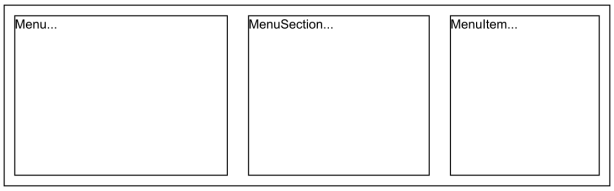


Figure 1.7: The Menu, MenuSection, and MenuItem classes

R2: Each branch will offer a menu with various sections and items.

Order

The `Order` class is identified by an ID and has a status that can be updated regarding adding or removing meals.

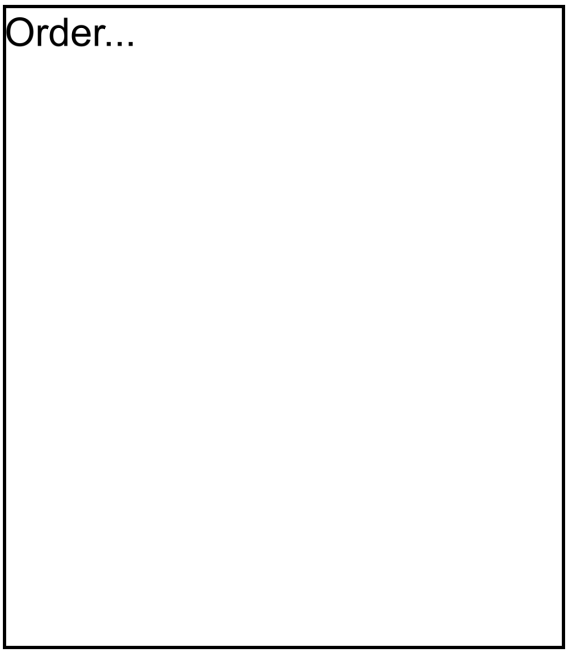


Figure 1.8: The Order class

R3: The server should be able to create an order for a table and add items for each person seated.
R4: Each person's order can consist of multiple items, each corresponding to a menu item.

Reservation

A `Reservation` is made by the receptionist for the customer. This class has a reservation ID, reservation time, count, customer information, status, notes, and check-in time.

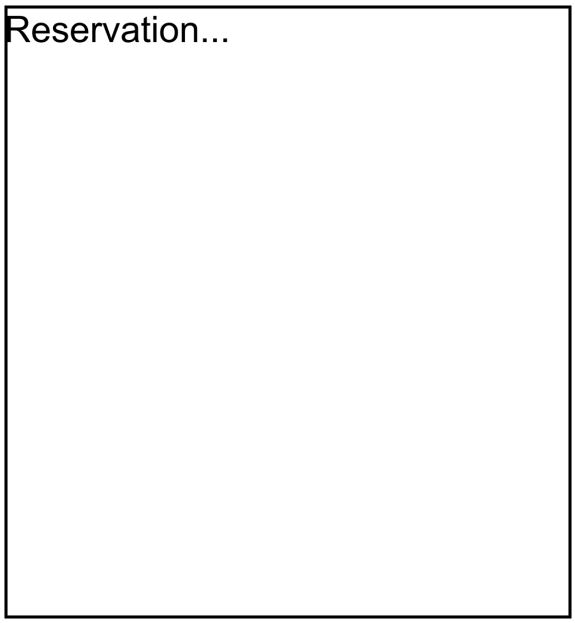


Figure 1.9: The Reservation class

R6: The system should allow for the reservation of tables.
R8: The system should allow customers to make and cancel reservations.
R9: The system should send notifications as the reservation time approaches.

Payment

`Payment` is an abstract class extended by `CreditCardPayment` and `CashTransaction` as its child classes. This class stores a payment ID, the total amount, and the date.

- `CreditCardPayment`: This derived class represents a credit card transaction.
- `CashTransaction`: This derived class represents a payment made in cash.

R10: Customers should be able to pay their bills with credit cards, checks, or cash.

Bill

The `Bill` class represents the total amount a customer has to pay based on the items ordered from the menu. This class stores an ID, the total amount to be paid, and the tax. Moreover, it records whether or not the bill was paid.
R10: Customers should be able to pay their bills with credit cards, checks, or cash.

Notification

A `Notification` is a message sent to a customer from the restaurant. Every `Notification` has a specific ID, the date it was sent, and the content the customer wrote.
R9: The system should send notifications as the reservation time approaches.

Seating chart and branch

`TableConfiguration`**:** This class represents the seating plan of a restaurant. A unique ID identifies it.
`Branch`**:** This class represents the branches of a restaurant. Every `Branch` has a specific name and location.

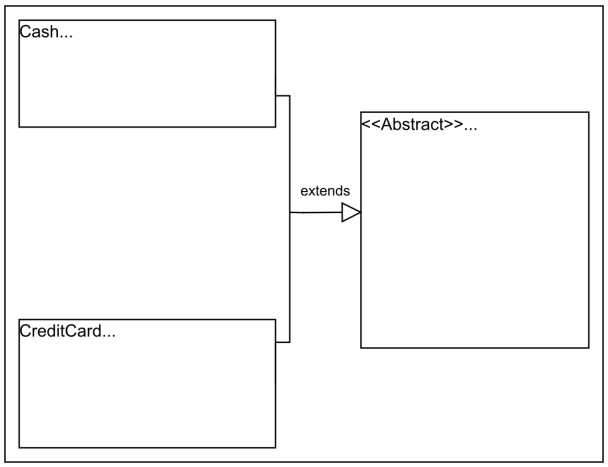


Figure 1.10: Payment and its derived classes

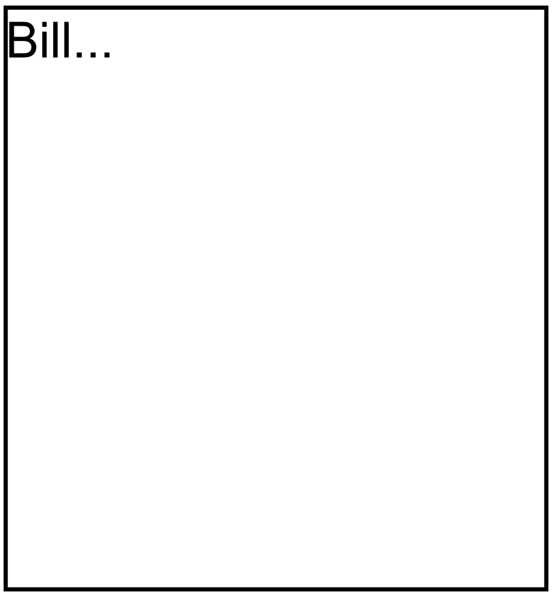


Figure 1.11: The Bill class

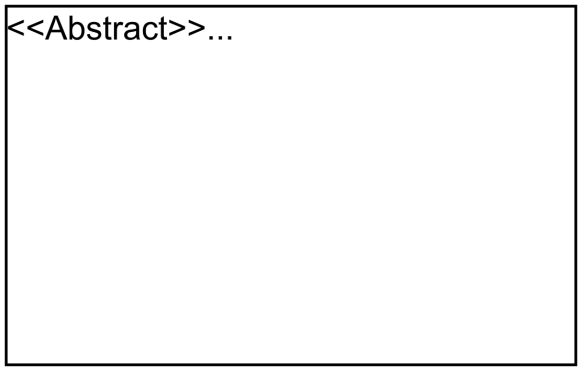


Figure 1.12: The Notification and its derived classes

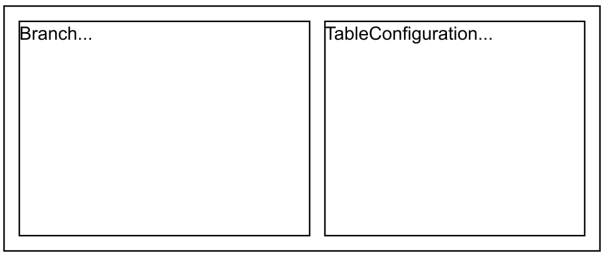


Figure 1.13: The SeatingChart, Branch, and Restaurant classes

R1: The restaurant can have multiple branches.
R2: Each branch will offer a menu with various sections and items.
R11: Each branch may have different configurations of tables.

Enumerations and custom data types

`TableStatus`**:** This enumeration checks whether a table is reserved, occupied, or free.
`OrderStatus`**:** This enumeration keeps track of a customer's order status.
`PaymentStatus`**:** This enumeration keeps track of an order's payment status by the customer.
`ReservationStatus`**:** This enumeration represents the reservation status of a table for a customer.

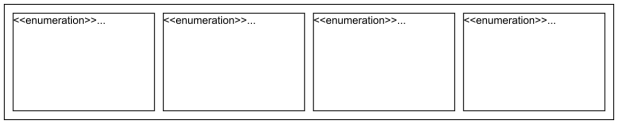


Figure 1.14: Enums in the restaurant management system

`Address`: This custom data type represents the address of the restaurant's branch or a person.

4.2 Relationship between the classes

Now, we'll discuss the relationships between the classes we have defined above in our restaurant management system.

Figure 1.15: Class diagram of the Address custom data type

Association

The class diagram has the following association relationships:

One-way Association

- The `Waiter`, `Manager`, and `Receptionist` class has a one-way association with the `Branch`.
- The `MealItem` class has a one-way association with `MenuItem`.
- The `Waiter` class has a one-way association with `Order`.
- The `Order` class has a one-way association with `Table`.
- The `Receptionist` class has a one-way association with `Reservation`.
- The `Reservation` class has a one-way association with `Table`.

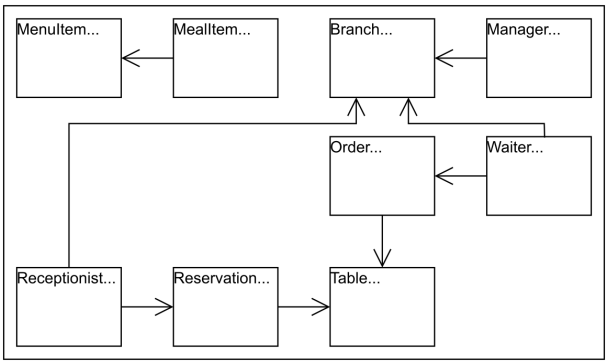


Figure 1.16: The one-way association relationship between classes

Two-way Association

- The `Branch` and `Menu` classes are associated with each other.
- The `Customer` and `Reservation` classes are associated with each other.
- The `Reservation` and `Notification` classes are associated with each other.
- The `Payment` and `Bill` classes are associated with each other.
- The `Bill` and `Order` classes are associated with each other.

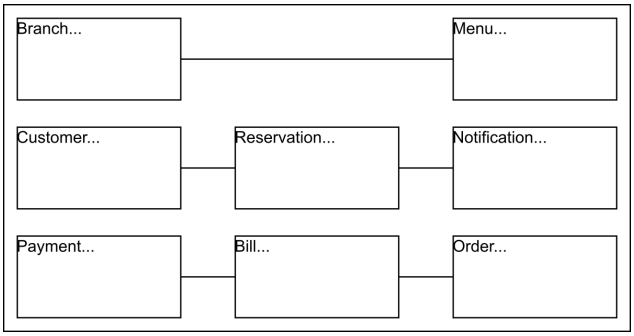


Figure 1.17: The two-way association relationship between classes

Composition

The class diagram has the following composition relationships.

- The `Branch` class is composed of `TableConfiguration`.
- The `Meal` class is composed of `MealItem` and `Table`.
- The `Order` class is composed of `Meal`.
- The `MenuSection` class is composed of `MenuItem`.
- The `Menu` class is composed of `MenuSection`.

Inheritance

The following classes show an inheritance relationship:

- All the classes, `Manager`, `Waiter`, `Receptionist`, and `Customer`, extend the `Person` class.
- `Payment` class is extended by `CreditCardPayment` and `CashTransaction`.

  Note: We have already discussed the inheritance relationship between classes in the component section above one by one.

4.3 Class diagram for the restaurant management system

This section outlines the multiplicity (cardinality) relationships between the main classes in our restaurant management system. We explain the allowed number of instances on each side for each relationship and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Here's the complete class diagram for the restaurant management system:

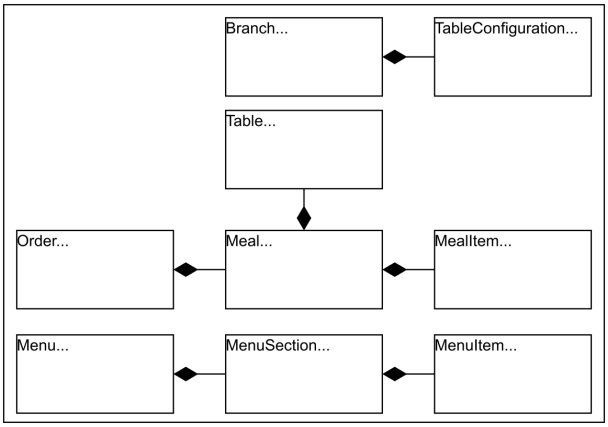


Figure 1.18: The composition relationship between classes

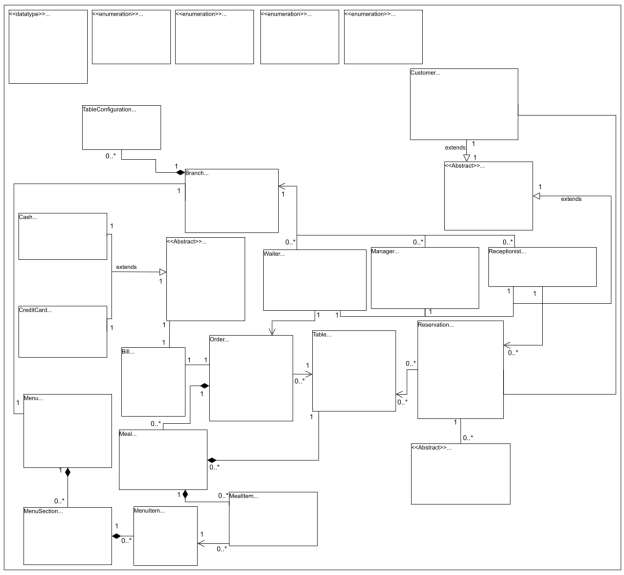


Figure 1.19: The class diagram of the restaurant management system

Source	Target	Multiplicity	Reason
Address	Customer	1–1	A customer has exactly one address.
Customer	Reservation	1–0..*	A customer can make multiple reservations.
Customer	Order	1–0..*	A customer can place multiple orders.
Reservation	Table	0..*–1	A reservation can be made for multiple tables.
Table	Branch	1–1	A table belongs to exactly one branch.
Branch	TableConfiguration	1–0..*	A branch can have multiple table configurations.
TableConfiguration	Table	1–0..*	A table configuration can relate to multiple tables.
Order	Table	1–0..*	An order can be associated with multiple tables.
Order	Bill	1–0..*	An order can generate multiple bills.
Bill	Payment	1–1	A bill can have one payment.
Bill	Cash	1–0..1	A bill can be paid in cash.
Bill	CreditCard	1–0..1	A bill can be paid via credit card.
Waiter	Order	1–0..*	A waiter can handle multiple orders.
Waiter	Table	1–0..*	A waiter can handle multiple tables.
Order	Meal	1–0..*	An order can contain multiple meals.
Meal	MenuItem	1–0..*	A meal can have multiple menu items.
MenuSection	MenuItem	1–0..*	A menu section can contain multiple menu items.
Menu	MenuSection	1–0..*	A menu can have multiple menu sections.
Menu	TableConfiguration	1–0..*	A menu can be linked to multiple table configurations.
Manager	Branch	1–0..*	A branch may have multiple managers.
Receptionist	Reservation	1–0..*	A receptionist can manage multiple reservations.
Reservation	Notification	0..*–1	A reservation can have multiple notifications.
Waiter	Branch	1–0..*	A branch may have multiple waiters.
Receptionist	Branch	1–0..*	A branch may have multiple receptionists.

Table 1.1: Entity Relationships and Multiplicities in a Restaurant Management System

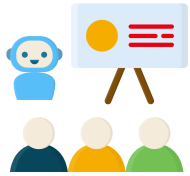
4.4 Design pattern

The following design patterns have been used in the class diagram:

- We know that a restaurant can accept different types of payments, such as cash or credit cards. To model this behavior, we can use the Strategy design pattern.
- Every restaurant payment follows a general process: prepare the bill, initiate the transaction, and finalize payment. To model this behavior, we can use the Template Method design pattern.
- A restaurant menu is organized into sections (like starters, main courses, and desserts), containing multiple items. The Composite design pattern can be used to model this hierarchical structure.
- We know that employees and customers in a restaurant share some common attributes like name, email, and address, but have different roles and responsibilities. We can use inheritance and polymorphism to model this shared behavior with specialization.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



4.6 Additional requirements

The interviewer can introduce some additional requirements in the given restaurant management system, or they can ask some follow-up questions. Let 's see examples of the additional requirements:
Discount: A discount will be applied to the payment depending on special events such as the New Year, an anniversary, a branch opening, etc. The class diagram provided below shows the relationship of `Discount` with the `Payment` class:

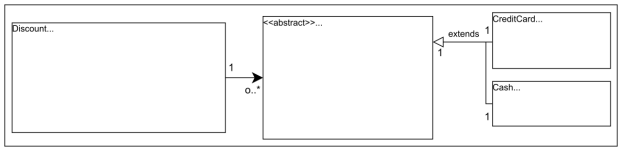


Figure 1.20: The relationship of the Discount class with the Payment class

We can update the `Payment` class to include an `applyDiscount()` method that can be invoked during the transaction process. This method would check if a discount is applicable based on the payment's creation date and, if so, apply the discount to the total amount.
We have completed the class diagram of the restaurant management system according to the requirements. In the next lesson, let's design its sequence diagram.

5 Sequence Diagram for the Restaurant Management System

Sequence diagrams are a great way to understand the interactions between different entities and objects in the system. There can be different sequence diagrams that we can create for our restaurant management system. In this lesson, we will create sequence diagrams for the following two interactions:

- A customer makes a reservation and receives notification.
- **Sequence challenge:** Customer places the order and pays the bill.

5.1 Making a reservation

The sequence diagram for making a reservation should have the following actors and objects that will interact with each other:

- **Actors:** `Customer` and `Receptionist`
- **Objects:** `TableReservation` and `Notification`

Here are the steps in the reservation interaction:

1. The customer initiates the process by searching for tables for a specific time and date.
2. The system returns a list of available tables to the customer.
3. The customer requests a reservation by selecting a table, time, and date.
4. The receptionist confirms and saves the reservation in the system.
5. The reservation is recorded and stored under the customer's details.
6. As the reservation time approaches, the system sends an automated notification to the customer.
7. The customer receives the notification about their upcoming reservation.

Based on the flow above, the sequence diagram for making a reservation in a restaurant management system is given below:

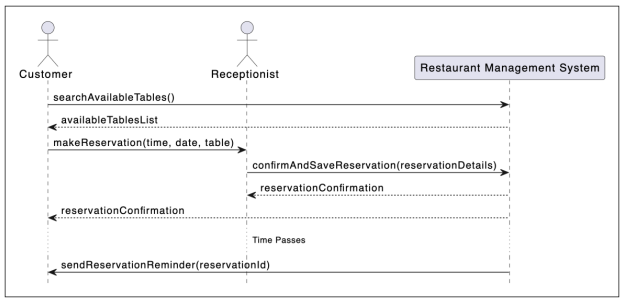


Figure 1.21: The sequence diagram for making a reservation

5.2 Sequence challenge: Customer places the order and pays

Let's complete a sequence diagram for the placing order and payment process.
A skeleton of the sequence diagram, given that the payment is successful, is provided below:

Notice that the arrows in the diagram above are numbered from 1 to 8. Below are the messages between the actor(s) and object(s). Can you rearrange the messages below in the correct sequence they should appear in the skeleton of the sequence diagram above?

> **Note:** If you're stuck, click the "Show Solution" button.

Alternatively, click the "Show complete diagram" button to view the complete sequence diagram.
In the next lesson, we'll create activity diagrams for the restaurant management system.

6 Activity Diagram for the Restaurant Management System

Activity diagrams are a great way to visualize the flow of messages from one activity to another in the system. There can be different activity diagrams that we can create for our restaurant management system. In this lesson, we'll create activity diagrams for the following two activities:

- Activities for the walk-in customer to place an order
- **Activity challenge:** Generate a bill and process the payment.

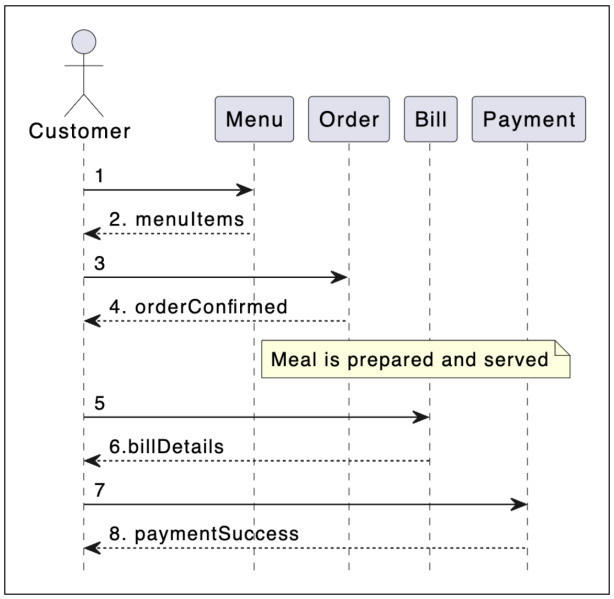


Figure 1.22: A sequence diagram of a customer placing the order and paying the bill

6.1 Place the order

The following are the states and actions that will be involved in this activity diagram.

States

Initial state: A customer arrives at the restaurant and asks for a table.
Final state: The customer successfully placed a food order.

Actions

The customer arrives at the restaurant's reception and requests a table. The receptionist checks for table availability. If a table is available, the customer is seated. If no table is available, the customer waits until one becomes free.
Once the customer is seated, the server approaches with the menu. The customer reviews the menu and decides what to order. The waiter then takes the customer's order.
Based on the order above, the activity diagram of the table booking and the food order is shown below:

6.2 Activity challenge: Billing and payment

Let's create an activity diagram for a customer to pay the bill.
A skeleton of the activity diagram is given below.

Notice that the actions in the diagram above are numbered from 1 to 10. The slots shown below represent the activities. Can you rearrange the slots below in the correct order they should appear in the activity diagram above?

Note: If you're stuck, click the "Show Solution" button.

Component type 'Permutation' not yet supported.

Alternatively, click the "Show complete diagram" button below to see the complete activity diagram.

We've looked at some of our restaurant management system's activity diagrams. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

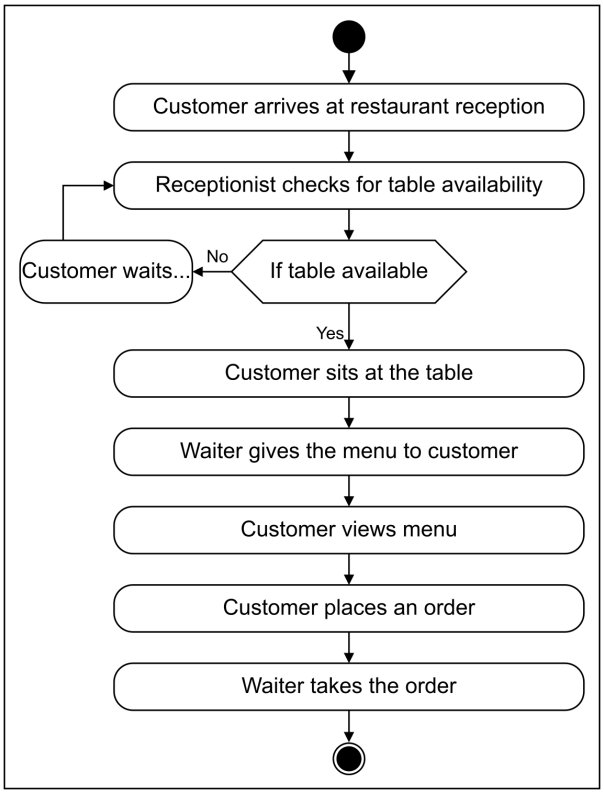


Figure 1.23: The activity diagram for placing the order

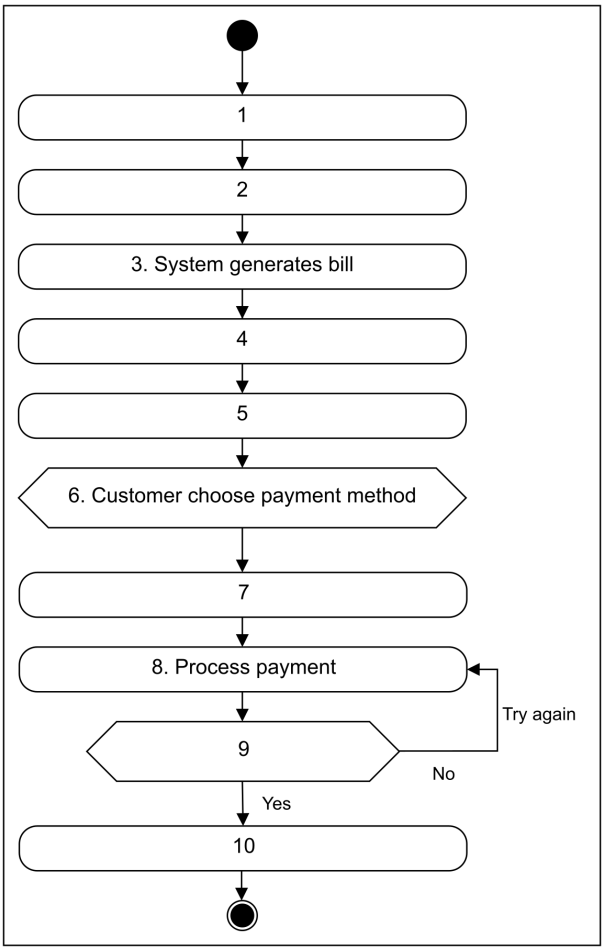


Figure 1.24: The activity diagram for order cancellation

7 Code of Restaurant Management System

Using various UML diagrams, we've covered different aspects of the restaurant management system and observed the attributes attached to the problem. Let us now explore the more practical side of things, where we will work on implementing the restaurant management system using multiple languages. This is usually the last step in an object-oriented design interview process.
We have chosen the following languages to write the skeleton code of the different classes present in the RMS:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click **Executable Code: Restaurant Management System**.

7.1 Restaurant management system classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Enumerations and custom data type

The following code defines the enumeration and custom data type used in the restaurant management system.
`PaymentStatus:` This enumeration keeps track of an order's payment status by the customer.
`TableStatus:` This enumeration keeps track of the table's status.
`OrderStatus:` This enumeration keeps the customer's order status in check.
`ReservationStatus:` This enumeration represents the reservation status of a table.
`Address:` This custom data type represents the address of the restaurant's branch or a person.

> **Note:** JavaScript does not support enumerations, so that we will use the `Object.freeze()` method as an alternative. This method freezes an object and prevents further modifications.

Definition of enums and custom data type

```
// Enumerations
enum PaymentStatus {
  Unpaid,
  Declined,
  Paid
}

enum TableStatus {
  Free,
  Reserved,
  Occupied
}

enum OrderStatus {
  Received,
  Preparing,
  Complete
```

```
}

enum ReservationStatus {
  Requested,
  Confirmed,
  CheckedIn,
  Canceled
}

// Custom Address data type class
public class Address {
  private int zipCode;
  private String address;
  private String city;
  private String state;
  private String country;
}
```

Person

The `Person` class is an abstract class storing personal details such as `name`, `email`, `password`, `address`, and `phone`. The following classes extend it:

- `Customer`: Represents a restaurant customer. Customers can interact with the menu, reservations, orders, and payments.
- `Receptionist`: Has a method `confirmAndSaveReservation(reservationDetails)` to handle table bookings on behalf of customers.
- `Manager`: This role also manages the menu, table configuration.
- `Waiter`: This class has a method called `placeOrder()` to record customer food orders. Additionally, waiters can fetch order lists, replace meal items, and generate bills.

The definitions of these classes are provided below:

Person and its derived classes

```
public abstract class Person {
  private String name;
  private String email;
  private String password;
  private Address address;
  private String phone;
}

public class Customer extends Person {

  public Table<list> searchAvailableTables();
  public boolean makeReservation(time, date, table);
  public Order placeOrder(selectedItems);
  public boolean cancelReservation();
  public bool payBill(paymentMethod);
  public Bill requestBillDetails();
}

public class Receptionist extends Person {
  public boolean confirmAndSaveReservation(reservationDetails);
  public List<Table> searchAvailableTables(Date date, Time time);
}

public class Manager extends Person {
  public TableConfiguration addTableConfig();
  public Menu updateMenu();
```

```
}


public class Waiter extends Person {
  public boolean addOrder(selectedItems);
  public Menu viewMenu();
  public List<Order> fetchOrderList(List<Order> allOrders);
  public Bill generateBill(orderid);
  public List<Table> searchAvailableTables(Date date);
}
```

Table

The `Table` class has a unique ID. It stores the table status, location identifier, and maximum capacity. Reservations can be added to a table using the `addReservation()` method. The definition of the class is provided below:

The Table class

```
public class Table {
  private int tableID;
  private TableStatus status;
  private int maxCapacity;
  private int locationIdentifier;
  private int numberOfSeats;

  public boolean isTableFree();
  public boolean addReservation();
  public static List<Table> search(int capacity, Date startTime);
}
```

Meal and meal item

The `MealItem` class is identified by a unique ID and stores the quantity of a `MenuItem`. It allows updating the quantity using the `updateQuantity()` method. A `MealItem` can be associated with a `Meal`, representing the meal ordered for a specific seat at a table. A `Meal` maintains a list of `MenuItem` objects and allows adding items through the `addMealItem()` method. The definitions of these classes are provided below:

The Meal and MealItem classes

```
public class MealItem {
  private int mealItemID;
  private int quantity;
  private MenuItem menuItem;

  public boolean updateQuantity(int quantity);
}

public class Meal {
  private int mealID;
  private Table table;
  private int seatNumber;
  private List<MenuItem> menuItems;

  public boolean addMealItem(MealItem mealItem);
}
```

Menu, menu section, and menu item

The `Menu` class represents the menu of a restaurant. A unique ID identifies it and maintains a list of `MenuSection`, where the `MenuSection` class represents a menu's different sections. A `MenuSection` lists `MenuItem` in which an item's price can be updated. The definitions of these classes are provided below:

The Menu, MenuSelection, and MenuItem classes

```
public class Menu {
  private int menuID;
  private String title;
  private String description;
  private double price;
  private List<MenuSection> menuSections;

  public boolean addMenuSection(MenuSection menuSection);
}

public class MenuSection {
  private int menuSectionID;
  private String title;
  private String description;
  private List<MenuItem> menuItems;

  public boolean addMenuItem(MenuItem menuItem);
}

public class MenuItem {
  private int menuItemID;
  private String title;
  private String description;
  private double price;

  public boolean updatePrice(double price);
}
```

Order and reservation

An `Order` has several meals and is assigned to a waiter for a specific table. Meals can be updated in an `Order` using the `addMeal()` and `removeMeal()` methods.
The `Reservation` class represents a table reservation that stores the reservation time, people count, status, check-in time, and customer information. It also maintains a list of notifications for the customer.
The definitions of these classes are provided below:

The Order, Kitchen, and Reservation classes

```
public class Order {
  private int OrderID;
  private OrderStatus status;
  private Date creationTime;
  private Meal[] meals;
  private Table table;
  private Waiter waiter;

  public boolean addMeal(Meal meal);
  public boolean removeMeal(Meal meal);
  public boolean replaceMealItem(Meal oldItem, Meal newItem);
}


public class Reservation {
  private int reservationID;
  private Date timeOfReservation;
  private int peopleCount;
```

```
  private ReservationStatus status;
  private String notes;
  private Date checkInTime;
  private Customer customer;
  private Table[] tables;
  private List<Notification> notifications;

  public boolean updatePeopleCount(int count);
}
```

Payment and bill

The `Payment` class is an abstract class with the `Check` and `CreditCard` classes as its child classes. This takes the `PaymentStatus` enum to keep track of the payment status. The `Bill` class represents the bill for a customer's order. The definitions of these classes are provided below:

Payment and its derived classes

```
// Payment is an abstract class
public abstract class Payment {
    private int paymentID;
    private Date creationDate;
    private double amount;
    private PaymentStatus status;

    public abstract void processPayment();
    public abstract void updateTableStatus(Table table);
}

public class CreditCard extends Payment {
    private String nameOnCard;
    private int zipcode;

    public void processPayment();
    public void updateTableStatus(Table table);
}

public class Cash extends Payment {
    private double cashTendered;

    public void processPayment();
    public void updateTableStatus(Table table);
}

public class Bill {
    private int billId;
    private float amount;
    private float tax;
    private boolean isPaid;
}
```

Notification

The `Notification` class is a class responsible for sending notifications. The implementation of this class is shown below:

The Notification class

```
public class Notification {
    private int notificationId;
    // The Date data type represents and deals with both date and time.
```

```
    private Date createdOn;
    private String content;

    public void sendReminderNotification(Person person);
}
```

Table configuration and branch

The `TableConfiguration` class stores the seating plan for a `Branch` of the Restaurant. The `Branch` of a restaurant has a specific name, address, and table configuration. The definitions of these classes are provided below:

The SeatingChart, Branch, and Restaurant classes

```
public class TableConfiguration {
  private int tableConfigID;
  private byte[] tableConfigImage;
}

public class Branch {
  private String name;
  private Address location;
  private Menu menu;
  private TableConfiguration tableConfig;
}
```

7.2 Executable code: Restaurant management system

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of a restaurant management system. The main driver code resides in `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the codes.

What does this code show

- **System initialization:** The simulation begins by creating the core components of the restaurant system: a customer named Alice, a waiter, a receptionist, and manager, a menu containing items like pizza and pasta, a table with availability, and the foundational reservation, order, bill, and payment structures. This setup creates a realistic baseline for simulating restaurant operations in sequence.

- **Scenario 1 (Customer makes a reservation):** The customer interacts with the receptionist to reserve a table. A reservation is created and confirmed, and the table status is updated to reflect the booking.

- **Scenario 2 (Customer places an order):** After being seated, the customer orders pizza and pasta. The waiter wraps these items in a meal, adds them to the order, and serves them.

- **Scenario 3 (Bill generation and payment):** The waiter generates a bill, and the customer pays using a credit card. The system processes the payment and updates the table's status to "Free."

  Hint: Try customizing the code!
  This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Executable Code: Restaurant Management System

```java
import java.util.*;

public class Driver {
    public static void main(String[] args) {
        System.out.println("=== Restaurant Management System Simulation ===
");

        // SYSTEM SETUP
        Address address = new Address(12345, "123 Main St", "Springfield", "IL", "USA");

        Waiter waiter = new Waiter("John", "john@restaurant.com", "waiter123", address, "555-9876");
        Receptionist receptionist = new Receptionist("Emma", "emma@restaurant.com", "rec123",
            address, "555-4567");
        Manager manager = new Manager("Olivia", "olivia@restaurant.com", "mgr456", address, "
            555-8888");

        Customer customer = new Customer("Alice", "alice@example.com", "cust001", address, "555-1122
            ");
        System.out.println("[1] Customer registered: " + customer.getName());

        // MENU SETUP
        MenuItem pizza = new MenuItem(1, "Pizza", "Cheesy pizza", 12.0);
        MenuItem pasta = new MenuItem(2, "Pasta", "Creamy Alfredo pasta", 10.0);
        MenuSection section = new MenuSection(1, "Main", "Main Course", new ArrayList<>(Arrays.
            asList(pizza, pasta)));
        Menu menu = new Menu(1, "Dinner Menu", "Evening specials", 0.0, new ArrayList<>(Arrays.
            asList(section)));
        System.out.println("[2] Menu and items initialized.");

        // TABLE SETUP
        Table table = new Table(101, TableStatus.Free, 4, 1, 4);
        System.out.println("[3] Table created. ID: " + table.getTableID());

        // ----------------------------------------
        // SCENARIO 1: Reservation Flow (Customer  Receptionist  Table)
        // ----------------------------------------
        System.out.println("
--- SCENARIO 1: Table Reservation ---");

        Reservation reservation = new Reservation(
                1,
                new Date(), // reservation time
                2,
                ReservationStatus.Requested,
                "Near window",
                null,
                customer,
                new Table[]{table},
                new ArrayList<>()
        );

        boolean isConfirmed = receptionist.confirmAndSaveReservation(reservation);
        if (isConfirmed) {
            reservation.setStatus(ReservationStatus.Confirmed);
            table.setStatus(TableStatus.Reserved);
            System.out.println("Reservation confirmed by receptionist for table " + table.getTableID
                ());
        }

        // ----------------------------------------
        // SCENARIO 2: Order Flow (Customer  Waiter  Meal  Order)
        // ----------------------------------------
        System.out.println("
--- SCENARIO 2: Placing an Order ---");

        MealItem order1 = new MealItem(1, 1, pizza);
        MealItem order2 = new MealItem(2, 1, pasta);

        Meal meal = new Meal(1, table, 1, new ArrayList<>());
        meal.addMealItem(order1);
        meal.addMealItem(order2);
```

```
        Meal[] meals = {meal};
        Order order = new Order(1, OrderStatus.Received, new Date(), meals, table, waiter);
        System.out.println("Order created for table " + table.getTableID() + " with items: Pizza and
             Pasta");

        // ----------------------------------------
        // SCENARIO 3: Billing and Payment (Waiter  Bill  Payment  Table Status)
        // ----------------------------------------
        System.out.println("
--- SCENARIO 3: Billing & Payment ---");

        float subtotal = (float)(pizza.getPrice() + pasta.getPrice());
        float tax = subtotal * 0.1f;
        Bill bill = waiter.generateBill(order.getOrderID());

        float total = bill.getAmount() + bill.getTax();
        CreditCard payment = new CreditCard(1, new Date(), total, PaymentStatus.Unpaid, "Alice",
            12345);

        // waiter.initiatePayment(bill, payment);
        payment.processPayment();
        payment.updateTableStatus(table);

        System.out.println("Bill paid: $" + total + " | Status: " + payment.getStatus());
        System.out.println("Table status after payment: " + table.getStatus());

        System.out.println("
=== Simulation Completed ===");
    }
}
```

7.3 Wrapping up

In this chapter, we've explored the complete design of the restaurant management system. We 've also looked at how a basic restaurant management system can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing a Restaurant Management System

Now that you've completed the restaurant management system case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

8.1 Key requirements

Below are the core functional and non-functional requirements for the restaurant management system:

1. The restaurant can have multiple branches, each with unique menus and table configurations.
2. Waiters can create and update orders for tables, adding multiple items per person.
3. Customers and receptionists can search for tables and make, modify, or cancel reservations.
4. The system can provide real-time information on available tables for walk-in customers.

5. The system notifies customers as their reservation time approaches.
6. Customers can pay bills using credit cards or cash; servers process payments.
7. Managers can update branch menus and table layouts.
8. The system automatically updates table statuses after reservations and payments.

8.2 System actors

The following are the main actors who interact with the restaurant management system:

- **Customer:** Browses menu, makes or cancels reservations, places orders, and pays bills.
- **Receptionist:** Searches and manages customer table reservations, receives notifications, and modifies reservations.
- **Server:** Assists with order taking, management, bill generation, and payment processing.
- **Manager:** Manages menu sections/items and table configuration for each branch.

8.3 Important classes and relationships

The table below summarizes the key classes, their responsibilities, and primary relationships:

8.4 Design highlights

This section captures the main design ideas behind the system.

8.5 Design approach

The system uses a bottom-up approach, beginning with atomic units (table, seat, menu item) and building up to meals, orders, reservations, and finally, full branch and restaurant management.

8.6 Design principles

The design demonstrates the following SOLID principles:

- **Single Responsibility principle (SRP):** Each class has a focused purpose (e.g., `Waiter` manages orders, `Notification` handles alerts).
- **Open/Closed principle (OCP):** Extending base classes allows adding new payment types, menu sections, or notification methods.
- **Liskov Substitution principle (LSP):** Derived classes (e.g., `Customer`, `Waiter`) can replace their parents without affecting system behavior.
- **Interface Segregation principle (ISP):** Functionalities like reservations, ordering, and payments are handled by separate classes, avoiding "fat" interfaces.
- **Dependency Inversion principle (DIP):** High-level modules depend on abstractions (`Payment`, `Notification`) rather than concrete implementations.

Class	Description	Important Relationships
Person	Abstract base class for all staff and customers.	Extended by Customer, Waiter, Receptionist, Manager.
Customer	Makes reservations, places orders, and pays bills.	Inherits from Person; associated with Reservation, Order, Address.
Waiter	Takes orders and processes payments for tables.	Inherits from Person; associated with Order, Branch, Table.
Receptionist	Manages table reservations.	Inherits from Person; associated with Reservation, Branch.
Manager	Manages menu and table layout per branch.	Inherits from Person; associated with Branch.
Branch	Represents a restaurant location.	Associated with TableConfiguration, Menu, Manager, Waiter, Receptionist.
Table	Represents a table with seats and status.	Associated with Branch, Reservation, Order.
TableConfiguration	Represents the seating chart for a branch.	Composed in Branch; associated with Table.
Menu	Contains menu sections and menu items.	Associated with Branch; composed of MenuSection.
MenuSection	A category of menu items (e.g., appetizers, entrees).	Composed of MenuItem; part of Menu.
MenuItem	Individual food/drink item with details and price.	Associated with MealItem; part of MenuSection.
Meal	Represents the food ordered per person at a table.	Composed of MealItem, Table; part of Order.
MealItem	A specific menu item within a meal, with quantity.	Associated with MenuItem; part of Meal.
Order	An order placed for a table includes meals for each seat.	Composed of Meal; associated with Table, Waiter, Bill.
Bill	Represents the total amount for an order, including tax.	Associated with Order, Payment.
Payment	Handles payments for bills.	Parent to CreditCardPayment, CashTransaction; associated with Bill.
Reservation	Table reservation with time, status, and customer details.	Associated with Table, Customer, Receptionist, Notification.
Notification	Sends alerts to customers (e.g., reservation reminders).	Associated with the Reservation.

Table 1.2: Restaurant Management System Classes and Their Relationships

8.7 Design patterns

Several established patterns are used:

- **Strategy pattern:** This pattern is used for payment processing. The system supports different payment strategies (cash, credit card), allowing the payment process to be dynamically selected and extended.

- **Template Method pattern:** Each payment type (cash or credit card) follows the same overall process (prepare bill, initiate transaction, finalize payment), but can implement steps differently. This common flow with customizable steps is modeled with the template method pattern.

- **Composite pattern:** The restaurant menu is structured hierarchically: a menu contains multiple sections (like starters or desserts), each containing multiple items. This whole-part relationship is modeled using the composite pattern.

8.8 Object interactions

Here is how objects collaborate for core activities:

- **Making a reservation:**
  - Customer searches for available tables via the receptionist/system.
  - System returns available options; customer selects table, date, and time.
  - The receptionist confirms and records the reservation.
  - A reservation is associated with the customer and table.
  - As the reservation time nears, the system sends a notification to the customer.
- **Placing an order and payment:**
  - The customer is seated and reviews the menu.
  - Waiter takes the order, creates an `Order`, and adds meals and meal items for each seat.
  - Waiter generates a bill linked to the order.
  - Customer selects payment method; waiter processes via `CreditCardPayment` or `CashTransaction`.
  - System updates the table status to available after payment.
- **Modifying or canceling a reservation:**
  - Customer or receptionist updates reservation; system updates associated table and notifies customer as needed.

8.9 System workflow

This section describes the main user journeys and life cycle events:

- **Branch setup and management:** The manager configures each branch's table layouts and updates the menu and table configurations as required. This ensures every branch has an accurate seating plan and an up-to-date menu for customers and staff.
- **Reservation flow:** Customers or receptionists search for available tables by date and time to make, modify, or cancel reservations. As reservation times approach, the system automatically sends notifications to customers and updates table statuses accordingly.
- **Walk-in ordering and dining:** Customers request tables at the reception and are seated if available. The server provides the menu, takes orders, and assists the customer throughout their dining experience.
- **Order and billing process:** The waiter creates or updates orders for the table, adds meal items for each customer, and generates the bill after service. The bill reflects all items ordered and includes any applicable taxes.
- **Payment handling:** Customers pay the bill by cash or credit card, which the waiter processes after the bill is generated. The system records the payment and marks the table as available for future reservations or walk-ins.

- **System-driven updates:** The system sends automatic notifications to customers as their reservation time approaches and updates table statuses after reservation and payment events. This keeps all actors informed and the restaurant's table management accurate in real time.

9 Key Insights: The Restaurant Management System

Well done on progressing with your restaurant management system design! This lesson shares practical ways to sharpen your approach, spot and avoid common mistakes, and prepare for key interview questions. You 'll also find a quick quiz to check your understanding and related case studies to help you keep practicing.

9.1 Common mistakes

Even strong candidates often stumble over a few classic errors when tackling the restaurant management system design in an interview. Look out for the following pitfalls:

- **Unclear relationship between `Order` and `Bill`:** Some candidates treat `Order` and `Bill` as completely separate without showing that the `Bill` summarizes the cost for an `Order`. Connect them properly so payment and order items stay in sync.
- **Missing edge cases for reservations:** It's common to forget scenarios like last-minute cancellations or no-shows. A solid RMS should have a `ReservationStatus` enum (pending, confirmed, canceled) and send notifications to staff and customers when changes happen.
- **Overlooking branch-specific variations:** Many overlook that each branch can have different menus, seating charts, or services. Always design the `Branch` and `Restaurant` classes to support unique configurations per branch.
- **Ignoring clear table status tracking:** Candidates sometimes forget to handle real-time table availability properly. Ensure `TableStatus` tracks if a table is free, reserved, or occupied so that the receptionist can manage walk-ins and reservations accurately.
- **Forgetting discount handling:** It's easy to miss that payments may need discounts for events or promotions. Make sure your `Payment` or `Bill` class can handle applying discounts when needed.

9.2 Key interview tips

Keep these practical tips in mind to strengthen your restaurant management system design.

9.3 Test your understanding

Let's put your knowledge to the test. These tricky MCQs will challenge your grasp of RMS design details, OOD principles, and real-world constraints.

Quiz

Question 1: Why does the RMS use the Factory pattern for menu items?

- To store menu items in a database
- **[CORRECT]** To dynamically create menu items without binding to specific classes
  Explanation: The Factory pattern abstracts the creation logic for flexible item instantiation.

Tip	Explanation
Clarify branch-specific behavior.	Explain how your design supports multiple branches with unique menus, seating charts, and services. Show how Branch and Restaurant work together.
Highlight design patterns used.	Point out where Singleton (for DB), Command (for requests), Observer (for notifications), and Factory (for menu items) are applied.
Show bottom-up thinking.	Highlight that you start with the smallest entities (tables, seats, menu items) and build up to larger components, showing structured design thinking.
Show role generalization clearly.	Make it clear that the Person class generalizes Manager, Receptionist, and Waiter roles, demonstrating good use of inheritance and clear responsibility boundaries.
Handle edge cases upfront.	Mention tricky scenarios such as discounts, reservation changes, and partial payments to show you've considered real-world complications.

Table 1.3: Design and Explanation Tips for Restaurant Management System

- To enforce a single instance of menu items
- To notify customers about new menu items

Question 2: What relationship exists between the `Reservation` and `Notification` classes?

- Inheritance
- **[CORRECT]** Association
  Explanation: Reservations are associated with notifications to inform customers.
- Aggregation
- Generalization

Question 3: How does the RMS handle multi-branch configurations?

- Using inheritance between branches.
- **[CORRECT]** By associating each `Branch` with its own `Menu` and `TableConfiguration`.
  Explanation: Each branch maintains unique settings via its `Menu` and `TableConfiguration`.
- By storing all branches in a `Table` class.
- By defining a `Branch` inside the `Kitchen` class.

Question 4: Why should the `Order` class have a status attribute?

- **[CORRECT]** To update the kitchen and server on meal progress
  Explanation: The `OrderStatus` keeps staff informed on preparation stages.

- To store payment information
- To track the table location
- To handle seat reservations

Question 5: Suppose the restaurant adds a feature where chefs can "peek" at upcoming orders without removing them from the queue. Which pattern is most appropriate to notify the chef of new orders while keeping the queue intact?

- Factory pattern
- Singleton pattern
- **[CORRECT]** Observer pattern
  Explanation: The Observer pattern allows the `Kitchen` (subject) to notify subscribed `Chef` observers whenever a new `Order` is added, without altering or consuming the queue.
- Command pattern

9.4 Related case studies

Explore the following related case studies to reinforce similar OOD principles:

- **Designing a hotel management system:** Shares concepts like reservations, room or table availability, payments, and staff roles. It is great for practicing handling bookings and services.
- **Designing a movie ticket booking system:** Reinforces seat selection, reservation flows, payment handling, and cancellation edge cases, just like table bookings in a restaurant.
- **Designing an Amazon online shopping system:** Covers ordering, payments, discounts, and inventory, which helps think through order-to-payment pipelines.
- **Designing an online food delivery system:** This extends the RMS with delivery-specific flows. You'll see how orders are dispatched, tracked, and updated in real time. It's a valuable parallel for supporting multiple order fulfillment channels.
- **Designing ride sharing service:** Demonstrates how to handle dynamic resource allocation (similar to table assignments), real-time notifications, and payments, with the added complexity of tracking moving assets.